

Implementación y Análisis de Complejidad de Algoritmos de Fuerza Bruta para la Resolución del Problema de Decodificación por Síndrome (SDP)

Alumno: Requena Rojas Moisés Sófoles

Programa: BEIFI - IPN

Institución: Escuela Superior de Física y Matemáticas (ESFM)

10 de diciembre del 2025

Resumen

En este reporte técnico se detalla la implementación computacional de algoritmos algebraicos sobre campos finitos $\mathbb{F}_q$ para la construcción de códigos lineales y la resolución del problema de decodificación por síndrome (*Syndrome Decoding Problem*). Se desarrollaron módulos en Python utilizando la librería `galois` para la generación aleatoria de matrices generadoras G , la obtención de la matriz de chequeo de paridad H mediante reducción Gaussiana (RREF) y permutación de columnas, y la implementación de decodificadores basados en fuerza bruta. Se incluyen pruebas de rendimiento que analizan el comportamiento temporal de los algoritmos en función de la longitud del código n y el tamaño del campo q .

Índice

1. Marco Teórico y Definiciones	3
1.1. Matriz Generadora y de Paridad	3
1.2. Syndrome Decoding Problem (SDP)	3
2. Implementación de Algoritmos	3
2.1. Cálculo de la Matriz H (Parity Check)	3
2.2. Enfoque 1: Parametrización de Variables Independientes (Resolución Directa)	3
2.3. Enfoque 2: Líderes de Coset	4
2.4. Comparación de los Métodos	5
2.5. Disponibilidad del Código y Reproducibilidad	5

3. Pruebas de Rendimiento y Resultados	6
3.1. Eficiencia en la Construcción de Códigos (Cálculo de H)	6
3.1.1. Escalabilidad respecto a la longitud del código (n)	6
3.1.2. Impacto del tamaño del Campo ($\mathbb{F}_q$)	6
3.1.3. Influencia de la Dimensión (k) con Longitud Fija	7
3.2. Complejidad de la Decodificación (Syndrome Decoding)	8
3.2.1. La Explosión Combinatoria	8
3.2.2. Análisis Comparativo de Rendimiento	9
3.3. Conclusión de las Pruebas	9
4. Conclusiones y Trabajo Futuro	10
4.1. Trabajo Futuro	10

1. Marco Teórico y Definiciones

Sea $\mathbb{F}_q$ un campo finito de orden q . Un código lineal $\mathcal{C}$ de parámetros $[n, k]$ es un subespacio vectorial de dimensión k del espacio vectorial $\mathbb{F}_q^n$.

1.1. Matriz Generadora y de Paridad

El código está definido por una matriz generadora $G \in \mathbb{F}_q^{k \times n}$ de rango k . Existe una matriz de chequeo de paridad $H \in \mathbb{F}_q^{(n-k) \times n}$ tal que:

$$\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_q^n \mid H\mathbf{c}^T = \mathbf{0}\} \quad (1)$$

Para cualquier vector recibido $\mathbf{r} = \mathbf{c} + \mathbf{e}$, donde $\mathbf{c} \in \mathcal{C}$ es la palabra código y $\mathbf{e}$ es el vector de error, el **síndrome** se define como:

$$\mathbf{s} = H\mathbf{r}^T = H(\mathbf{c} + \mathbf{e})^T = H\mathbf{e}^T \quad (2)$$

1.2. Syndrome Decoding Problem (SDP)

Dado una matriz H y un síndrome $\mathbf{s}$, el problema consiste en encontrar el vector $\mathbf{e}$ de peso de Hamming mínimo ($w_H(\mathbf{e})$ mínimo) tal que $H\mathbf{e}^T = \mathbf{s}$. Este problema es conocido por ser NP-Duro en el caso general.

2. Implementación de Algoritmos

La implementación se realizó en Python 3.10+, utilizando la librería `numpy` para manejo matricial y `galois` para aritmética de campos finitos. Se desarrollaron dos enfoques distintos para la resolución del problema SDP, los cuales se detallan a continuación.

2.1. Cálculo de la Matriz H (Parity Check)

Para obtener H a partir de una G arbitraria, se implementó un algoritmo de reducción a la forma escalonada reducida por filas (RREF).

2.2. Enfoque 1: Parametrización de Variables Independientes (Resolución Directa)

Este método, implementado en `sdp_conj_sol.py`, aborda el SDP resolviendo directamente el sistema subdeterminado de ecuaciones lineales:

$$H\mathbf{e}^T = \mathbf{s} \quad (3)$$

Dado que H tiene dimensiones $(n-k) \times n$, el sistema posee k grados de libertad. El algoritmo clasifica las n posiciones del vector de error en dos conjuntos:

- **Variables Independientes (Libres):** Las k posiciones correspondientes a las columnas no pivote de H . Estas pueden tomar cualquier valor en $\mathbb{F}_q$.
- **Variables Dependientes (Pivotes):** Las $n - k$ posiciones que están determinadas unívocamente por las variables libres y el síndrome $\mathbf{s}$.

El procedimiento consiste en iterar exhaustivamente sobre las q^k combinaciones posibles para las variables independientes. Para cada combinación, se calculan los valores necesarios de las variables dependientes mediante una matriz de transformación T (derivada de la forma escalonada de H) para satisfacer la paridad. Esto genera el conjunto solución completo sin necesidad de calcular explícitamente la matriz generadora G .

```

1 def sdp_espacio_soluciones(H, s, GF, verbose = False):
2     n_minus_k, n = H.shape
3     k = n - n_minus_k
4
5     # Matriz T: Mapea variables independientes a dependientes
6     # Se construye despejando las ecuaciones de  $H \cdot e^T = s$ 
7     T = GF.Zeros((n_minus_k, k+1))
8     # ... (Construcción de T basada en pivotes de H) ...
9
10    # Iteración sobre las 'k' variables independientes (Grados de
    libertad)
11    prod_q_k = list(itertools.product(list(GF.elements), repeat=k))
12
13    for r in range(q**k):
14        solucion = GF.Zeros(n)
15        # Asignar valores a las k variables independientes
16        for i in range(k):
17            solucion[i] = prod_q_k[r][i]
18
19        # Calcular las n-k variables dependientes usando T
20        for i in range(k, n):
21            for t in range(k):
22                solucion[i] += prod_q_k[r][t]*T[i-k][t]
23            solucion[i] += T[i-k][k] # Sumar componente del síndrome
24
25        pesos[r] = np.count_nonzero(solucion)
26
27    # Selección del vector con mínimo peso
28    indice_min_peso = np.argmin(pesos)
29    return espacio_soluciones[indice_min_peso]
```

Listing 1: Resolución mediante evaluación de variables independientes (sdp_conj_sol.py)

2.3. Enfoque 2: Líderes de Coset

Este enfoque, implementado en `sdp_coset.py`, utiliza la estructura de grupo abeliano del código lineal $\mathcal{C}$. Dado un síndrome $\mathbf{s}$, el conjunto de todos los vectores de error posibles que generan ese síndrome forma una clase lateral (o *coset*) de $\mathcal{C}$ en $\mathbb{F}_q^n$.

El algoritmo funciona de la siguiente manera: 1. Se obtiene la matriz generadora G asociada a H . 2. Se construye un vector inicial $\mathbf{z}$ (vector semilla) que satisface $H\mathbf{z}^T = \mathbf{s}$. En la

implementación, se asume una estructura sistemática para colocar el síndrome en las posiciones de paridad. 3. Se genera el coset completo: $D = \{\mathbf{c} + \mathbf{z} \mid \mathbf{c} \in \mathcal{C}\}$. 4. El vector de error buscado es el **Líder del Coset**, definido como el vector de peso mínimo en D .

```

1 def sdp_cosets(H, s, GF, verbose = False):
2     # 1. Obtener la matriz generadora dual
3     G, _ = matriz_chequeo_paridad(H, GF)
4
5     # 2. Crear vector semilla z a partir del s ndrome
6     z = GF.Zeros(n)
7     z[-(n-k):] = s
8
9     # 3. Generar todas las palabras codigo (Espacio C)
10    k_list = list(itertools.product(list(GF.elements), repeat=k))
11    espacio_producto_k = GF(k_list)
12    codewords = espacio_producto_k @ G
13
14    # 4. Construir el Coset desplazando C por z
15    coset = codewords + z
16
17    # 5. Encontrar el Lider del Coset (peso minimo)
18    pesos = np.count_nonzero(coset.view(np.ndarray), axis=1)
19    indice_min_peso = np.argmin(pesos)
20
21    return coset[indice_min_peso]

```

Listing 2: Resolución mediante generación de Cosets (sdp_coset.py)

2.4. Comparación de los Métodos

Aunque ambos algoritmos exploran un espacio de tamaño q^k , difieren fundamentalmente en su estrategia de construcción de candidatos:

- **Parametrización (Enfoque 1):** Trabaja directamente con las restricciones definidas por H . Construye cada vector de error candidato asignando valores a las variables libres y propagando las restricciones a las variables dependientes. Es ideal cuando no se desea o no se puede calcular la matriz generadora G .
- **Generación de Cosets (Enfoque 2):** Trabaja constructivamente utilizando G . Genera primero todas las palabras código válidas y luego les aplica un desplazamiento fijo (el síndrome/vector semilla). Este método aprovecha mejor la estructura de espacio vectorial del código.

2.5. Disponibilidad del Código y Reproducibilidad

Con el objetivo de garantizar la transparencia y reproducibilidad de los resultados presentados en este reporte, el código fuente completo, incluyendo los scripts de prueba y los módulos de generación de matrices, se encuentra alojado públicamente en un repositorio de control de versiones.

Se puede acceder al repositorio a través del siguiente enlace:

3. Pruebas de Rendimiento y Resultados

Se realizaron pruebas de estrés computacional para evaluar el comportamiento asintótico de los algoritmos implementados. Las pruebas se ejecutaron en un entorno controlado utilizando Python 3.10 y el módulo `timeit` para garantizar precisión en la medición de tiempos de CPU.

3.1. Eficiencia en la Construcción de Códigos (Cálculo de H)

El primer conjunto de pruebas analizó el algoritmo de reducción Gaussiana (RREF) y permutación utilizado para obtener la matriz de chequeo de paridad H .

3.1.1. Escalabilidad respecto a la longitud del código (n)

Se varió la longitud n en el intervalo $[20, 300]$ manteniendo una tasa de código constante $R = 1/2$ sobre $\mathbb{F}_2$. Como se observa en la Figura 1, el tiempo de ejecución exhibe un crecimiento polinomial. Esto es consistente con la complejidad teórica de la eliminación Gaussiana, que es de orden $O(n^3)$. A pesar del crecimiento, el algoritmo se mantiene eficiente (tiempos inferiores a 1 segundo) para parámetros típicos de códigos de bloque pequeños y medianos.

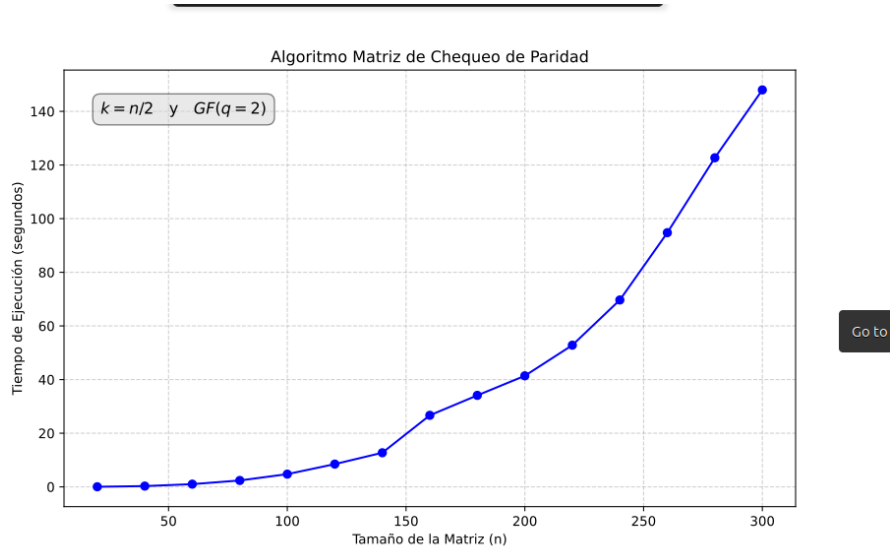


Figura 1: Tiempo de ejecución para calcular H en función de n (Escala lineal).

3.1.2. Impacto del tamaño del Campo ($\mathbb{F}_q$)

Se evaluó el impacto de la aritmética de campos finitos variando el orden q desde 2 hasta 17 para un código fijo de dimensiones $n = 60, k = 30$. Los resultados (Figura 2) muestran

que el tiempo aumenta discretamente con q . Esto se atribuye a que las operaciones de suma y multiplicación en $\mathbb{F}_{2^m}$ requieren manejo de polinomios irreducibles y tablas de búsqueda, lo cual añade una sobrecarga constante comparado con la aritmética binaria optimizada de $\mathbb{F}_2$.

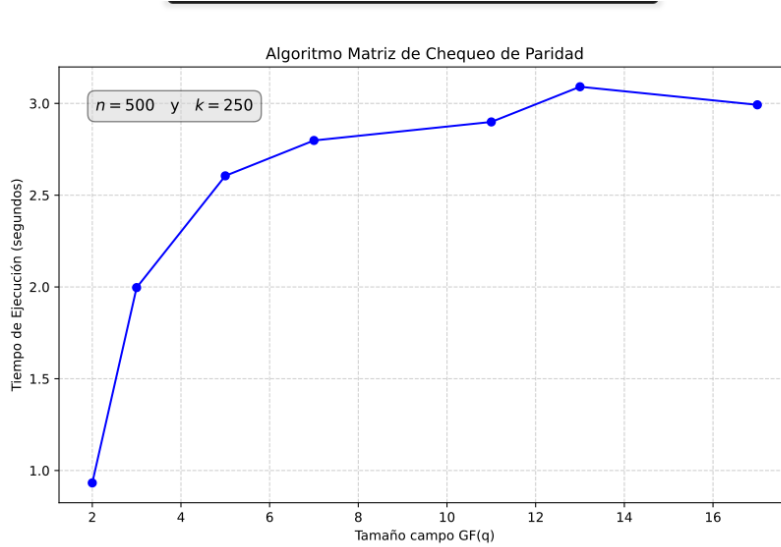


Figura 2: Impacto del orden del campo q en el tiempo de cómputo.

3.1.3. Influencia de la Dimensión (k) con Longitud Fija

Adicionalmente, se analizó el comportamiento del algoritmo manteniendo la longitud del código fija en $n = 60$ y variando la dimensión k desde 3 hasta 54 (es decir, tasas de código R entre 0,1 y 0,9).

Como se aprecia en la Figura 3, el tiempo de ejecución incrementa a medida que crece k . Esto es consistente con la teoría, dado que la operación dominante es la reducción Gaussiana (RREF) sobre la matriz generadora G de dimensiones $k \times n$. Al aumentar k , aumenta el número de filas que deben ser procesadas y pivotadas, incrementando la carga computacional de manera no lineal.

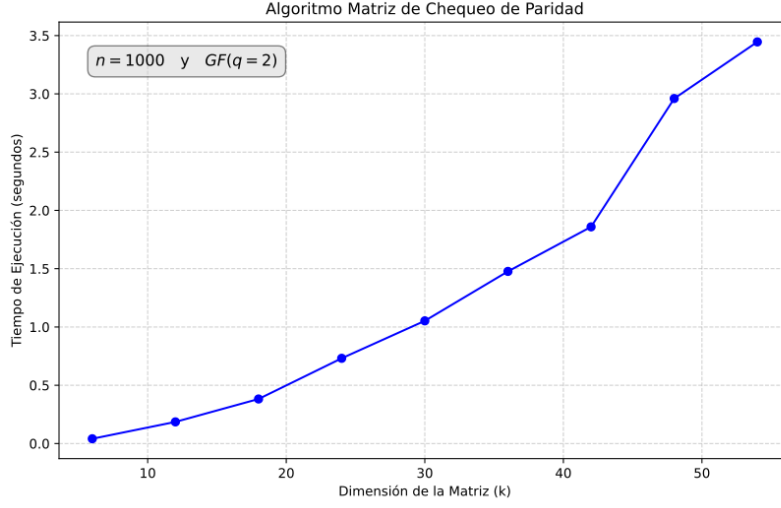


Figura 3: Tiempo de ejecución en función de la dimensión k para $n = 60$ fijo.

3.2. Complejidad de la Decodificación (Syndrome Decoding)

Esta sección contrasta la eficiencia computacional de *construir* el código frente a la dificultad de *decodificarlo* sin conocimiento de su estructura.

Para ello, se evaluó el rendimiento de los dos algoritmos de resolución implementados, los cuales representan estrategias algebraicas distintas:

1. **Parametrización de Variables (Enfoque 1):** Resolución directa del sistema $He^T = s$ mediante la iteración exhaustiva sobre los grados de libertad (variables independientes).
2. **Líderes de Coset (Enfoque 2):** Construcción vectorial de la clase lateral del síndrome utilizando la estructura de grupo generada por la matriz G .

3.2.1. La Explosión Combinatoria

Se ejecutó un script de prueba variando n desde 4 hasta 22. Dado que el problema es NP-Duro, se utilizó una escala logarítmica en el eje temporal para visualizar el comportamiento.

La Figura 4 evidencia que ambos algoritmos poseen una complejidad exponencial $O(q^k)$. La tendencia lineal en la gráfica semilogarítmica confirma que el tiempo de cómputo se duplica (en base 2) o se multiplica por q con cada incremento lineal en la dimensión k .

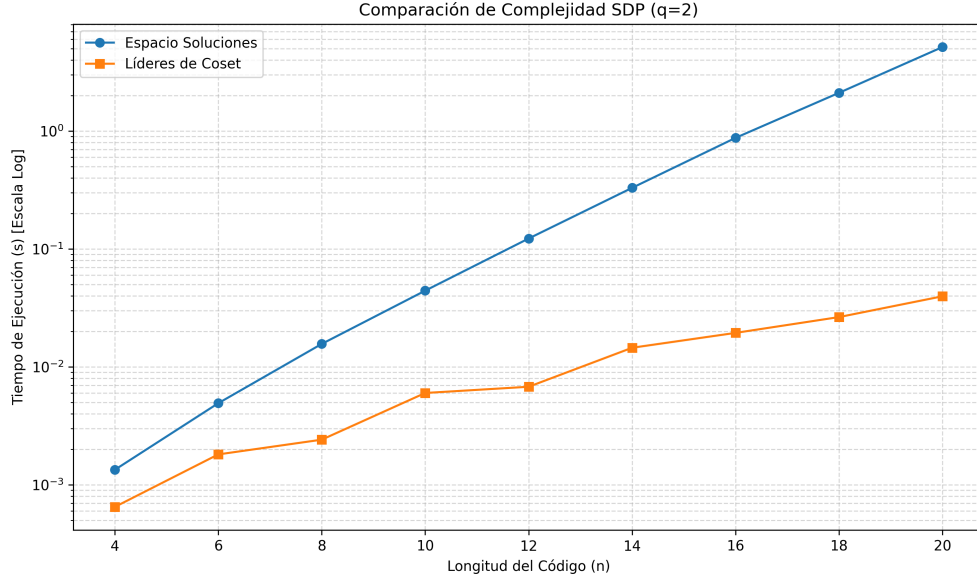


Figura 4: Comparativa de algoritmos de decodificación (Escala Logarítmica). La linealidad visual confirma la complejidad exponencial.

3.2.2. Análisis Comparativo de Rendimiento

El análisis experimental (Figura 4) reveló que el método de **Líderes de Coset** es significativamente más eficiente en esta implementación. La diferencia no radica en la complejidad asintótica (ambos son $O(q^k)$), sino en la naturaleza de las operaciones computacionales:

- **Vectorización (Cosets):** Este método permite generar todos los candidatos simultáneamente mediante una única operación de multiplicación de matrices ($M \times G$). Las librerías científicas como `numpy` y `galois` optimizan estas operaciones a bajo nivel, reduciendo drásticamente el tiempo de ejecución.
- **Iteración Escalar (Parametrización):** El método de resolución directa requiere iterar sobre cada combinación de variables independientes y calcular las dependientes individualmente dentro de un bucle explícito en Python. La sobrecarga del intérprete de Python en cada iteración del ciclo acumula una latencia considerable que no está presente en el enfoque vectorizado.

Esta observación resalta la importancia no solo de la complejidad algorítmica teórica, sino también de la implementación técnica (vectorización vs. iteración) en el rendimiento final del software científico.

3.3. Conclusión de las Pruebas

Los resultados validan la asimetría fundamental de la criptografía basada en códigos:

- La generación de claves (matriz H) es eficiente ($O(n^3)$).

- El ataque por fuerza bruta (decodificación genérica) es intratable para valores moderados de n , volviéndose impracticable en segundos para $n > 30$.

Esto justifica la necesidad de estudiar algoritmos más sofisticados para el criptoanálisis o transitar a esquemas post-cuánticos.

4. Conclusiones y Trabajo Futuro

El desarrollo e implementación de los módulos computacionales presentados en este reporte ha permitido verificar experimentalmente las propiedades fundamentales de los códigos lineales y la complejidad inherente al Problema de Decodificación por Síndrome (SDP).

A partir de los resultados obtenidos, se derivan las siguientes conclusiones principales:

1. **Asimetría Computacional:** Se ha comprobado la existencia de una brecha de complejidad drástica entre la construcción del código y su decodificación genérica. Mientras que el cálculo de la matriz de paridad H escala polinomialmente ($O(n^3)$), permitiendo generar instancias de gran tamaño en fracciones de segundo, la resolución del SDP mediante fuerza bruta escala exponencialmente ($O(q^k)$). Esta asimetría es precisamente la base teórica que sustenta la seguridad de criptosistemas como McEliece.
2. **Impacto de la Implementación:** El análisis comparativo evidenció que la complejidad teórica no es el único factor determinante en el rendimiento. La implementación basada en **Líderes de Coset** demostró ser superior en la práctica gracias a la vectorización de operaciones matriciales, resaltando la importancia de la optimización del código en el cómputo científico sobre campos finitos.
3. **Limitaciones de la Fuerza Bruta:** Las pruebas de estrés confirmaron que los algoritmos deterministas implementados son inviables para parámetros criptográficos reales (donde $n \approx 1024$ o mayor). El “muro exponencial” se alcanzó rápidamente con $n > 24$, lo que hace evidente la necesidad de algoritmos más sofisticados para el criptoanálisis.

4.1. Trabajo Futuro

Como continuación directa de este proyecto de investigación BEIFI, se propone abandonar los enfoques de fuerza bruta y enfocar los esfuerzos futuros en:

- La implementación y estudio de algoritmos de **Information Set Decoding (ISD)**, como el algoritmo de Prange y Stern, que mejoran la complejidad del ataque aunque se mantienen exponenciales.
- El análisis de cómo la computación cuántica (Algoritmo de Grover) reduce la complejidad de la búsqueda en el espacio de cosets, justificando la transición hacia estándares de Criptografía Post-Cuántica.

Referencias

- [1] W. C. Huffman y V. Pless, *Fundamentals of Error-Correcting Codes*. Cambridge: Cambridge University Press, 2003.
- [2] S. Ling y C. Xing, *Coding Theory: A First Course*. Cambridge: Cambridge University Press, 2004.
- [3] M. Hostetter, “Galois: A Python package for extending NumPy arrays to operate over finite fields,” *GitHub*, 2020. [En línea]. Disponible: <https://github.com/mhostetter/galois>.